Chapter 1: Algorithm Analysis and Big-O

An algorithm is a finite sequence of steps that solves a problem. To compare algorithms we measure how their running time and memory use grow as the input size n grows. Big-O notation describes the upper bound of this growth and ignores constants and lower-order terms.

Common growth rates from fastest to slowest are: O(1) constant time, O(log n) logarithmic time, O(n) linear time, O(n log n) linearithmic time, O(n squared) quadratic time and O(2 to the power n) exponential time. Binary search is O(log n), a simple loop over an array is O(n), and nested loops over the same array are usually O(n squared).

Time complexity can be stated for the best case, average case and worst case. Unless stated otherwise, Big-O usually refers to the worst case. Space complexity measures the extra memory an algorithm needs besides its input.

Chapter 2: Arrays and Linked Lists

An array stores elements in contiguous memory, so access by index is O(1). However, inserting or deleting in the middle of an array is O(n) because the following elements must be shifted. The size of a static array is fixed when it is created.

A linked list is a chain of nodes, where each node holds a value and a pointer to the next node. Inserting or deleting at the head of a linked list is O(1), but finding an element by position is O(n) because you must walk through the nodes one by one. A doubly linked list also stores a pointer to the previous node, which makes backward traversal and deletion easier at the cost of extra memory.

Choose an array when you need fast random access and the size is known. Choose a linked list when you need frequent insertions and deletions and do not need random access.

Chapter 3: Stacks and Queues

A stack is a Last-In, First-Out (LIFO) structure. The two main operations are push, which adds an item on top, and pop, which removes the top item. Both are O(1). Stacks are used for the function call stack, the undo feature in editors, checking balanced brackets and evaluating postfix expressions.

A queue is a First-In, First-Out (FIFO) structure. The operations are enqueue, which adds an item at the rear, and dequeue, which removes the item at the front. Queues are used in task scheduling, printer queues and breadth-first search. A circular queue reuses empty space at the start of the array to avoid wasting memory.

A priority queue is a special queue where each element has a priority and the element with the highest priority is removed first, regardless of arrival order. It is commonly implemented with a heap, giving O(log n) insertion and removal.

Chapter 4: Trees and Binary Search Trees

A tree is a hierarchical structure made of nodes connected by edges. The top node is the root, nodes with no children are leaves, and the height of a tree is the length of the longest path from the root to a leaf. In a binary tree every node has at most two children, called left and right.

A binary search tree (BST) keeps its nodes ordered: for every node, all values in the left subtree are smaller and all values in the right subtree are larger. Search, insertion and deletion take O(log n) time on average, but O(n) in the worst case when the tree becomes skewed like a linked list. Balanced trees such as AVL trees and red-black trees keep the height at O(log n).

There are three depth-first traversals: inorder (left, node, right), preorder (node, left, right) and postorder (left, right, node). An inorder traversal of a BST visits the values in sorted order. Level-order traversal visits nodes level by level using a queue.

Chapter 5: Sorting Algorithms

Sorting arranges elements in a defined order. Bubble sort repeatedly swaps adjacent elements that are out of order; it is simple but runs in O(n squared) time. Selection sort and insertion sort are also O(n squared) in the worst case, although insertion sort is fast on nearly sorted data.

Merge sort uses the divide-and-conquer approach: it splits the array into halves, sorts each half recursively and merges the sorted halves. It always runs in O(n log n) time and is stable, but it needs O(n) extra space. Quick sort picks a pivot, partitions the array into smaller and larger elements and sorts the parts recursively. Its average time is O(n log n), but the worst case is O(n squared) when the pivot choices are poor.

A sorting algorithm is stable if equal elements keep their original relative order. Python's built-in sort uses Timsort, a hybrid of merge sort and insertion sort, which is stable and runs in O(n log n) in the worst case.

Chapter 6: Graphs and Shortest Paths

A graph is a set of vertices (nodes) connected by edges. Edges can be directed or undirected, and weighted or unweighted. A graph can be stored as an adjacency matrix, which uses O(V squared) space, or as an adjacency list, which uses O(V + E) space and is better for sparse graphs.

Breadth-first search (BFS) explores a graph level by level using a queue and finds the shortest path in an unweighted graph. Depth-first search (DFS) goes as deep as possible along each branch using a stack or recursion, and is used for cycle detection and topological sorting.

Dijkstra's algorithm finds the shortest paths from one source vertex to all other vertices in a graph with non-negative edge weights. It repeatedly picks the unvisited vertex with the smallest known distance, using a priority queue, and relaxes its outgoing edges. With a binary heap it runs in O((V + E) log V). It does not work correctly when edges have negative weights; Bellman-Ford is used in that case.